ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		В. В. Жукалин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

РЕАЛИЗАЦИЯ КЛИЕНТСКОЙ ЧАСТИ ВЕБ-ПРИЛОЖЕНИЯ: ФОТОГРАФИИ, МОДАЛЬНЫЕ ОКНА, БУРГЕР-МЕНЮ, СОСТОЯНИЕ ПОЛЬЗОВАТЕЛЯ И КАРТА

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		И.А. Лаврешин
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Разработать интерактивную клиентскую часть веб-приложения SignalBoard на Next.js и React: вывести фотографии предметных сущностей, реализовать доступное модальное окно, создать бургер-меню, связать интерфейс с учебным состоянием пользователя и отобразить объекты на карте с помощью JavaScript API Яндекс Карт.

2 Задание

Продолжить проект лабораторной работы № 2. Дополнить тип сигнала фотографией и координатами, подготовить локальные изображения в каталоге public и вывести их через next/image. Создать интерактивную галерею карточек, управлять выбранной карточкой через useState, реализовать модальное окно через createPortal с закрытием мышью и клавишей Escape и удержанием клавиатурного фокуса. Создать бургер-меню с активным маршрутом, показать разные действия для гостя и учебного пользователя, подключить JavaScript API Яндекс Карт, создать карту и метки по координатам, освобождать внешние ресурсы в cleanup-функциях useEffect и проверить сборку командами npm run lint и npm run build.

3 Ход выполнения задания

Проект SignalBoard – доска локальных сигналов для соседей («ищу», «отдаю», «предупреждаю»). Работа продолжает лабораторную работу № 2: маршруты App Router, корневой layout и учебный DemoAuthProvider сохранены, новый create-next-app не запускался. Реализован клиентский сценарий: пользователь открывает раздел «Галерея и карта», видит карточки сигналов с фотографиями, открывает подробное окно щелчком по карточке или по метке на карте, а после учебного входа может отметить сигнал как отслеживаемый. Навигация по разделам перенесена в бургер-меню.

Обновлённая структура каталога app показана на рисунке 1.

Структура каталога app

```
signalboard/
├── app/
│   ├── about/
│   │   └── page.tsx
│   ├── auth/
│   │   ├── login/
│   │   │   └── page.tsx
│   │   └── register/
│   │       └── page.tsx
│   ├── cabinet/
│   │   └── page.tsx
│   ├── data/
│   │   └── signals.ts
│   ├── gallery/
│   │   ├── _components/
│   │   │   ├── PhotoMap.module.css
│   │   │   ├── PhotoMap.tsx
│   │   │   ├── PhotoModal.module.css
│   │   │   ├── PhotoModal.tsx
│   │   │   ├── SignalsGallery.module.css
│   │   │   └── SignalsGallery.tsx
│   │   └── page.tsx
│   ├── providers/
│   │   └── DemoAuthProvider.tsx
│   ├── signals/
│   │   ├── [id]/
│   │   │   └── page.tsx
│   │   └── page.tsx
│   ├── start/
│   │   └── page.tsx
│   ├── ui/
│   │   ├── BurgerMenu/
│   │   │   ├── BurgerMenu.tsx
│   │   │   └── styles.module.css
│   │   ├── AppNav.tsx
│   │   └── FeatureCard.tsx
│   ├── globals.css
│   ├── layout.tsx
│   └── page.tsx
├── public/
│   ├── images/
│   │   └── places/
│   │       ├── give-chair.jpg
│   │       ├── lost-keys.jpg
│   │       └── water-outage.jpg
│   ├── file.svg
│   ├── globe.svg
│   ├── next.svg
│   ├── vercel.svg
│   └── window.svg
├── .env.example
└── package.json
```

Рисунок 1 – Обновлённая структура проекта

Тип NeighborSignal из файла app/data/signals.ts дополнен полями для фотографии и карты. Свойства предметной сущности приведены в таблице 1.

Таблица 1 – Свойства предметной сущности «сигнал»

Свойство	**Тип**	**Назначение**

id	string	Уникальный идентификатор, ключ списка и сегмент URL
title	string	Заголовок карточки и модального окна
summary	string	Краткое описание сигнала
category	string	Тип сигнала: ищу / отдаю / предупреждаю
location	string	Текстовое место во дворе или доме
image	string	Путь к локальному изображению в public/images/places
imageAlt	string	Альтернативный текст изображения
latitude	number	Широта метки на карте
longitude	number	Долгота метки на карте

Источники фотографий. Изображения lost-keys.jpg, give-chair.jpg и water-outage.jpg являются собственными учебными иллюстрациями, подготовленными для проекта SignalBoard, и хранятся локально в каталоге public/images/places. Сторонние фотографии с ограничениями лицензии не используются. Для каждого изображения задан осмысленный alt, описывающий содержание кадра.

Страница app/gallery/page.tsx осталась Server Component: она выводит заголовок и подключает клиентский компонент SignalsGallery. Директива «use client» стоит только у компонентов, которым нужны состояние, обработчики событий и браузерные API: SignalsGallery, PhotoModal, PhotoMap и BurgerMenu. Так в браузер отправляется JavaScript только для интерактивной части.

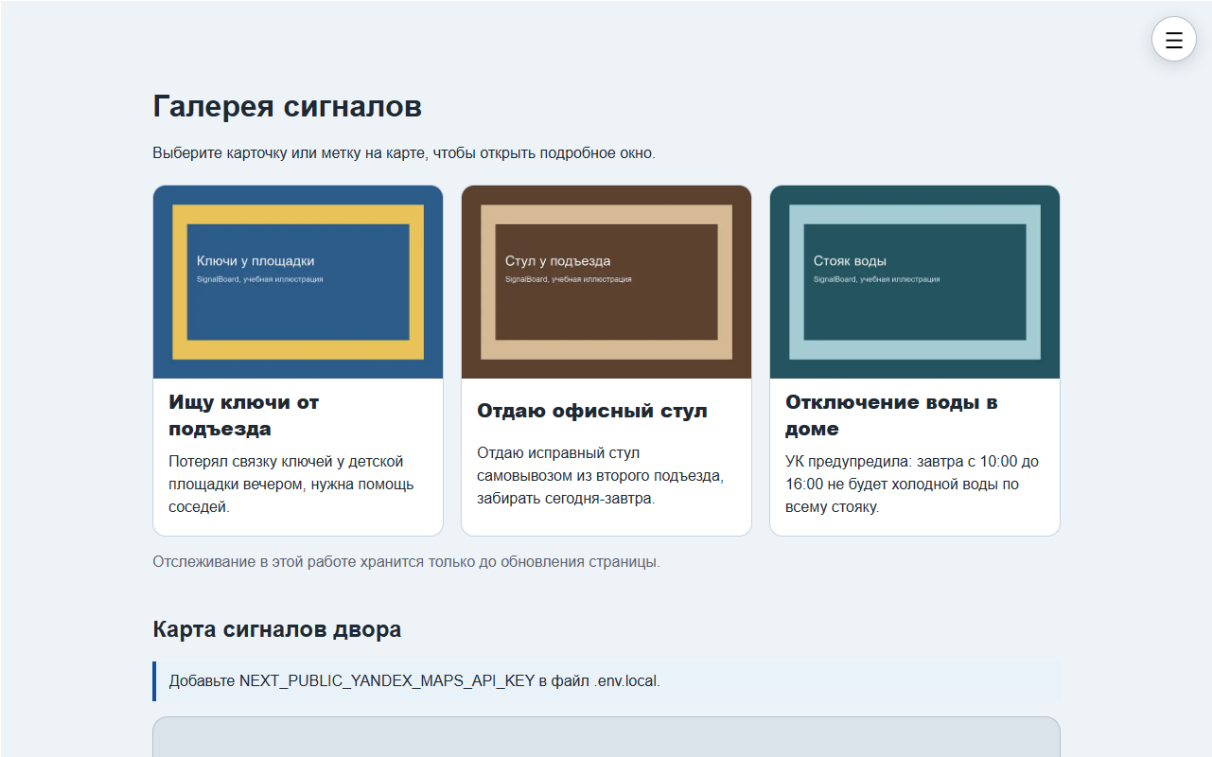


Рисунок 2 – Галерея сигналов /gallery

Используемые хуки и функции React:

– useState возвращает пару [значение, функция обновления]; в SignalsGallery он хранит выбранный сигнал (NeighborSignal | null), множество отслеживаемых id и элемент, на который нужно вернуть фокус;

– useEffect синхронизирует компонент с внешними системами: подписка на keydown для Escape, блокировка прокрутки документа, загрузка API карты; cleanup-функция снимает подписки, восстанавливает overflow и вызывает map.destroy() при закрытии окна или размонтировании;

– useRef хранит ссылки на DOM-элементы (диалог, кнопка закрытия, контейнер карты) и изменяемые значения, изменение которых не должно вызывать повторный рендер;

– createPortal из react-dom рендерит модальное окно в document.body, поэтому окно не обрезается overflow родителей и находится поверх страницы, оставаясь при этом частью дерева React и сохраняя доступ к контексту DemoAuthProvider.

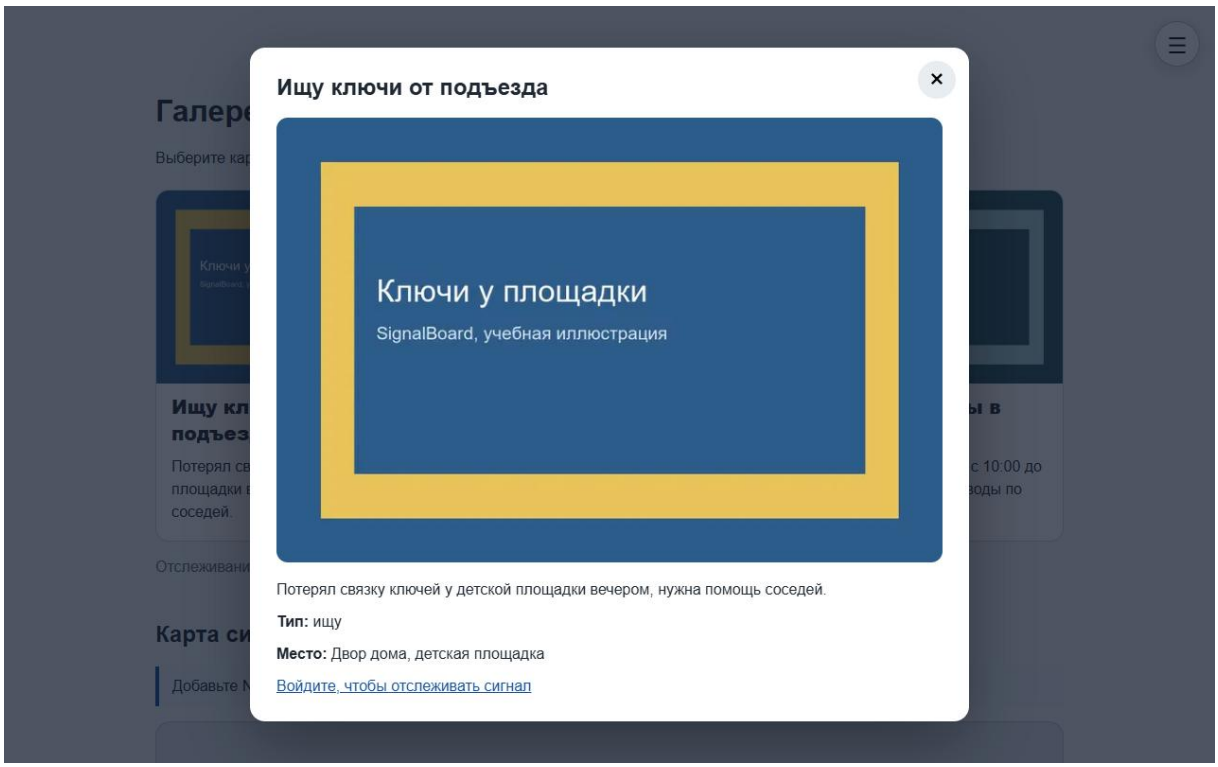


Рисунок 3 – Модальное окно сигнала (гость видит ссылку на вход)

Требования доступности модального диалога и их реализация приведены в таблице 2.

Таблица 2 – Требования доступности диалога

Требование	**Реализация в PhotoModal**
Роль диалога	role="dialog" и aria-modal="true" у контейнера
Доступное имя	aria-labelledby ссылается на заголовок h2 окна
Начальный фокус	При открытии фокус переводится на кнопку «Закрыть окно»
Цикл Tab / Shift+Tab	keepFocusInside переносит фокус с последнего элемента на первый и обратно
Закрытие Escape	Обработчик keydown на window, снимается в cleanup
Закрытие мышью	Кнопка × и щелчок только по затемнённому фону
Возврат фокуса	После закрытия фокус возвращается на карточку, открывшую окно
Прокрутка фона	overflow: hidden у documentElement на время открытия
Семантика карточки	Карточка – элемент button, доступный с клавиатуры

Бургер-меню app/ui/BurgerMenu/BurgerMenu.tsx заменило горизонтальную навигацию AppNav в корневом layout. Кнопка открытия

имеет aria-expanded и aria-controls, панель – role="dialog" и aria-modal. Активный раздел определяется через usePathname и отмечается aria-current="page". Меню закрывается по ссылке, клавишей Escape, кнопкой × и щелчком по фону; фокус возвращается на кнопку открытия. Состояние открытия привязано к маршруту, на котором меню открыли, поэтому после перехода по ссылке панель гарантированно закрыта.

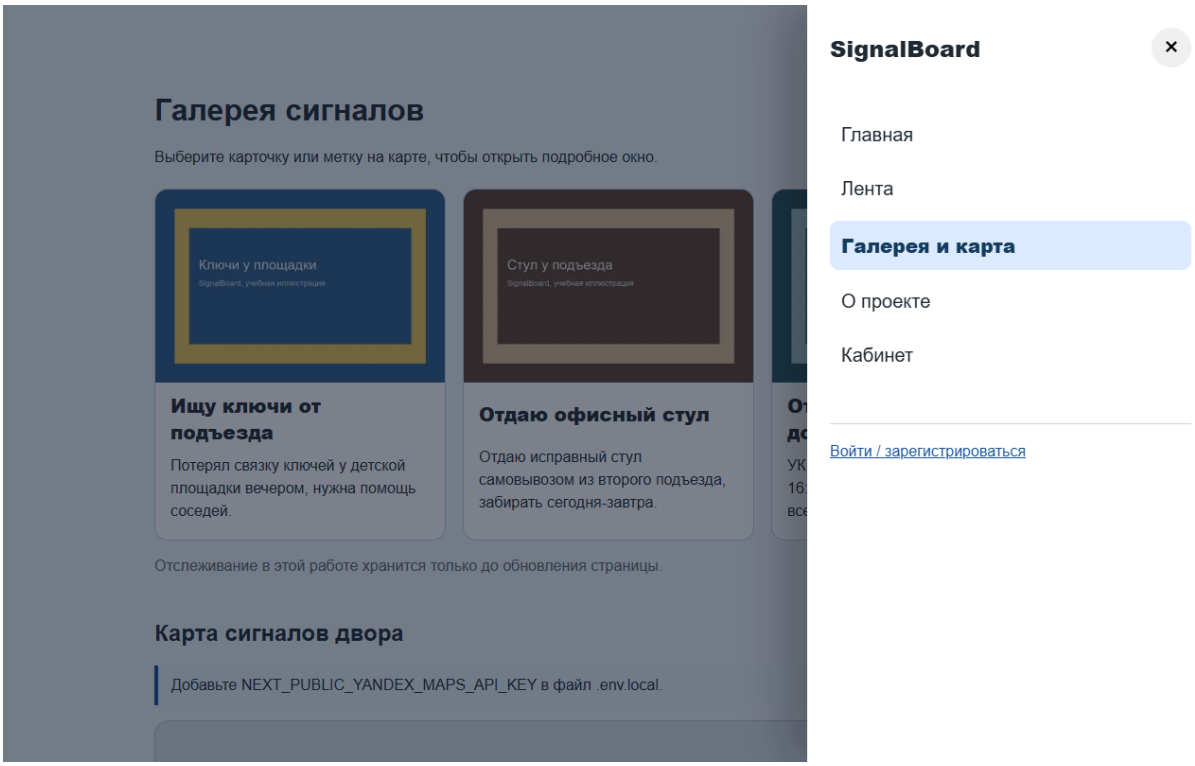


Рисунок 4 – Открытое бургер-меню, активный раздел «Галерея и карта»

Действие «Отслеживать сигнал» зависит от учебного состояния пользователя. Гость видит ссылку «Войдите, чтобы отслеживать сигнал» (рисунок 3). После учебного входа в модальном окне появляется кнопка, которая добавляет id сигнала в Set отслеживаемых (рисунки 5–6). Отслеживание хранится только в памяти вкладки и сбрасывается после обновления страницы.

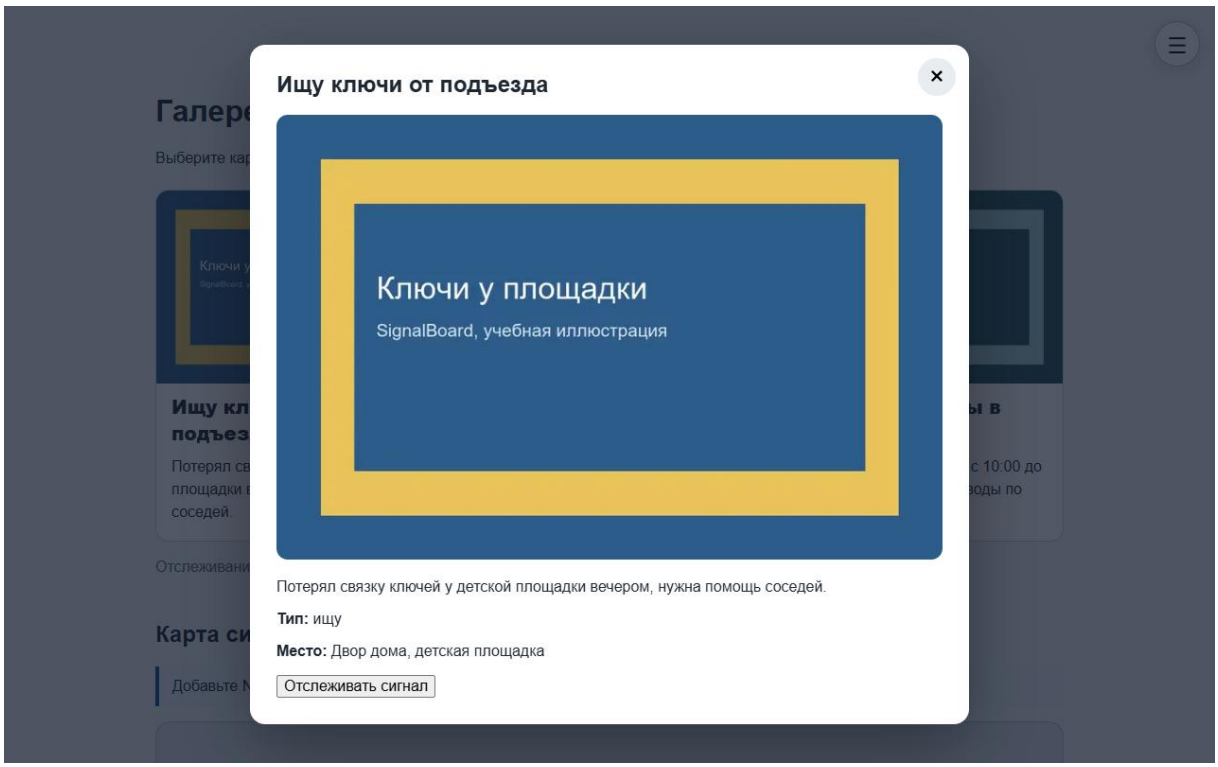


Рисунок 5 – Модальное окно после учебного входа

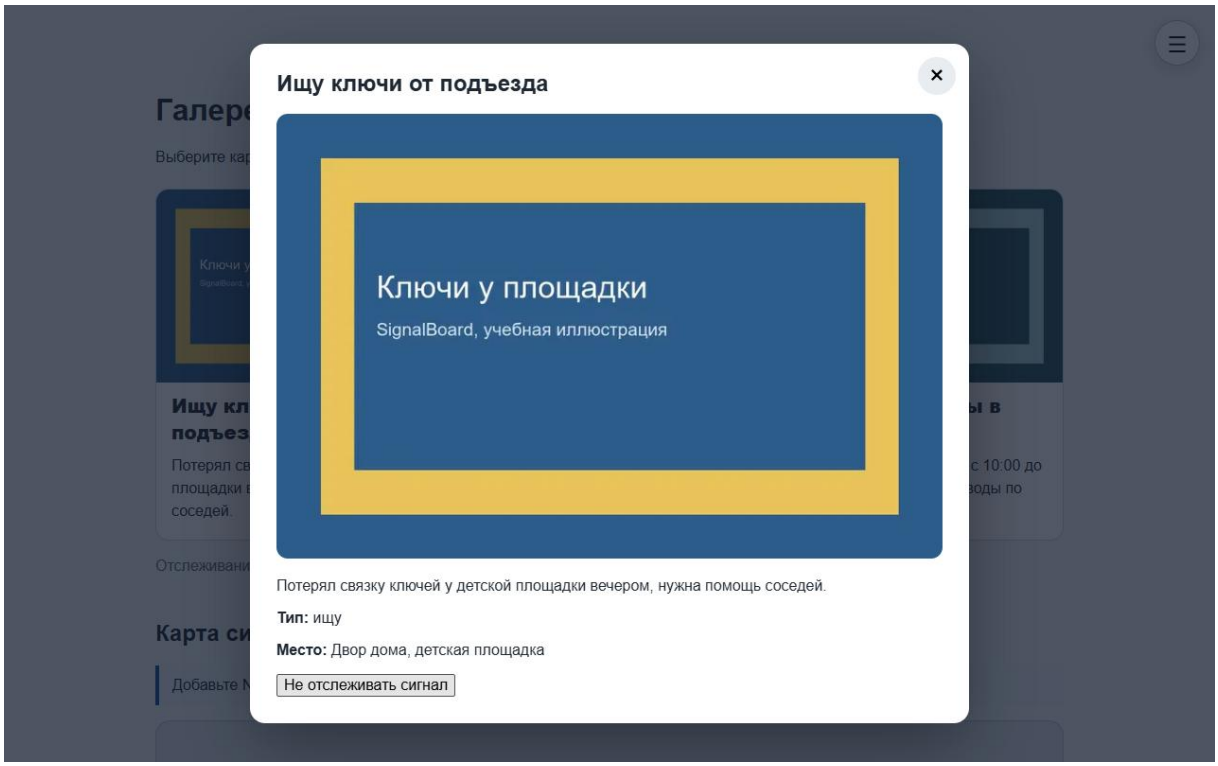


Рисунок 6 – Сигнал отмечен как отслеживаемый

Условная кнопка не является авторизацией. Она лишь скрывает элемент интерфейса, но не защищает данные: состояние хранится в браузере, пользователь может изменить его средствами разработчика, а защищённой

серверной операции ещё нет. Проверка прав будет выполнена на сервере в лабораторной работе № 6.

Карта реализована компонентом PhotoMap. Скрипт JavaScript API Яндекс Карт 2.1 подключается асинхронно один раз: элемент script создаётся динамически, а повторные вызовы получают тот же Promise. Скрипт загружается по сети после отрисовки страницы, поэтому объект window.ymaps появляется не сразу. Функция ymaps.ready вызывает переданный обработчик только после полной готовности API и DOM, и только после этого создаётся new ymaps.Map с центром в координатах первого сигнала. Для каждого сигнала создаётся ymaps.Placemark с hintContent и balloonContent, добавляется в map.geoObjects, а событие click открывает то же модальное окно, что и карточка. В cleanup вызывается map.destroy().

Ключ передаётся через переменную NEXT_PUBLIC_YANDEX_MAPS_API_KEY в файле .env.local. Префикс NEXT_PUBLIC_ означает, что значение встраивается в клиентский JavaScript при сборке и видно любому пользователю в исходном коде страницы, поэтому это публичный ключ с ограничением по HTTP Referer, а не серверный секрет. Настоящий ключ в отчёт не включается. Если ключ не задан, компонент не загружает API и показывает понятное сообщение (рисунок 7). Снимок карты с метками выполняется после записи собственного ключа (рисунок 8).

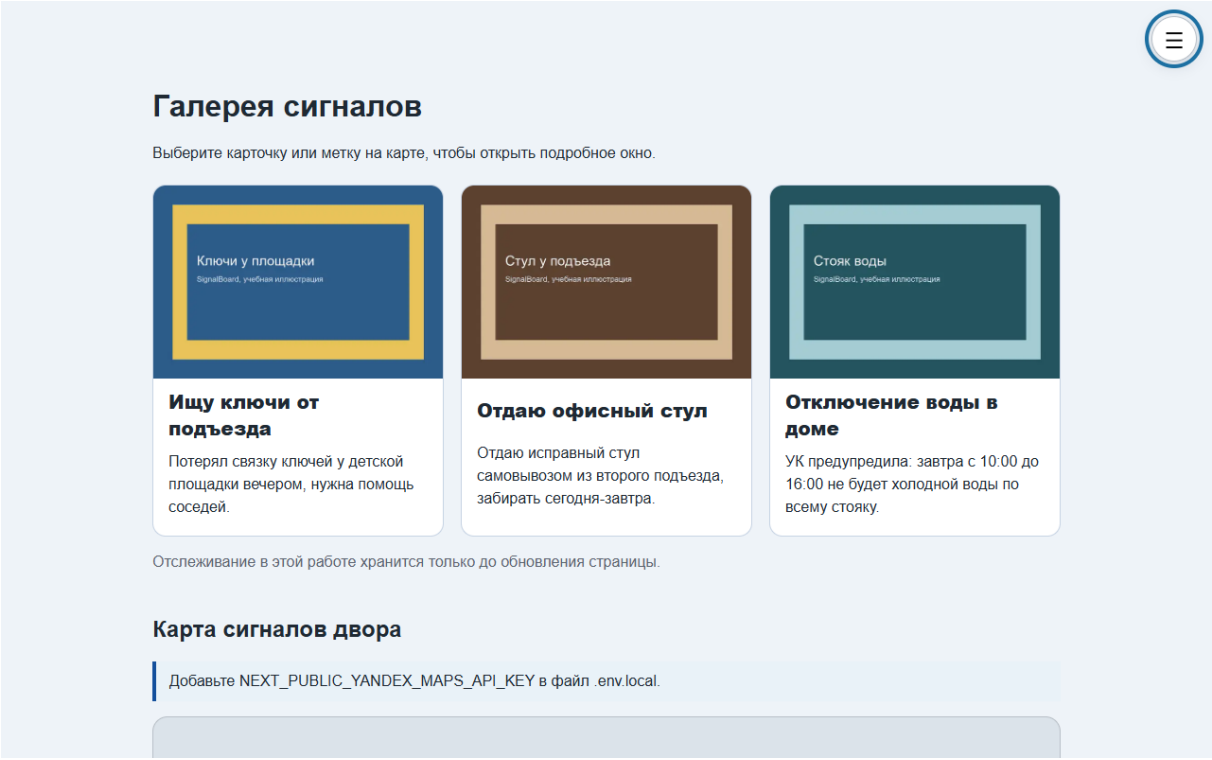


Рисунок 7 – Сообщение об отсутствии ключа карты

[ВСТАВИТЬ СКРИНШОТ: раздел «Карта сигналов двора» после записи своего ключа в .env.local и перезапуска npm run dev]

Рисунок 8 – Карта с метками сигналов

Проверка кода выполнена командами npm run lint и npm run build. ESLint ошибок не обнаружил, production-сборка завершилась успешно (рисунки 9–10). Функция openFromMap создаётся через useCallback, чтобы зависимость onSelect эффекта карты была стабильной и карта не пересоздавалась при каждом рендере.

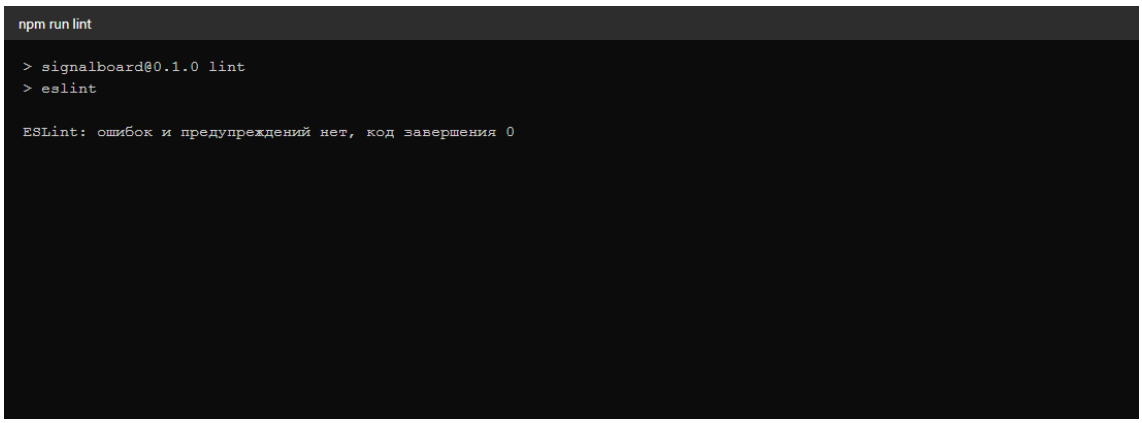


Рисунок 9 – Результат npm run lint

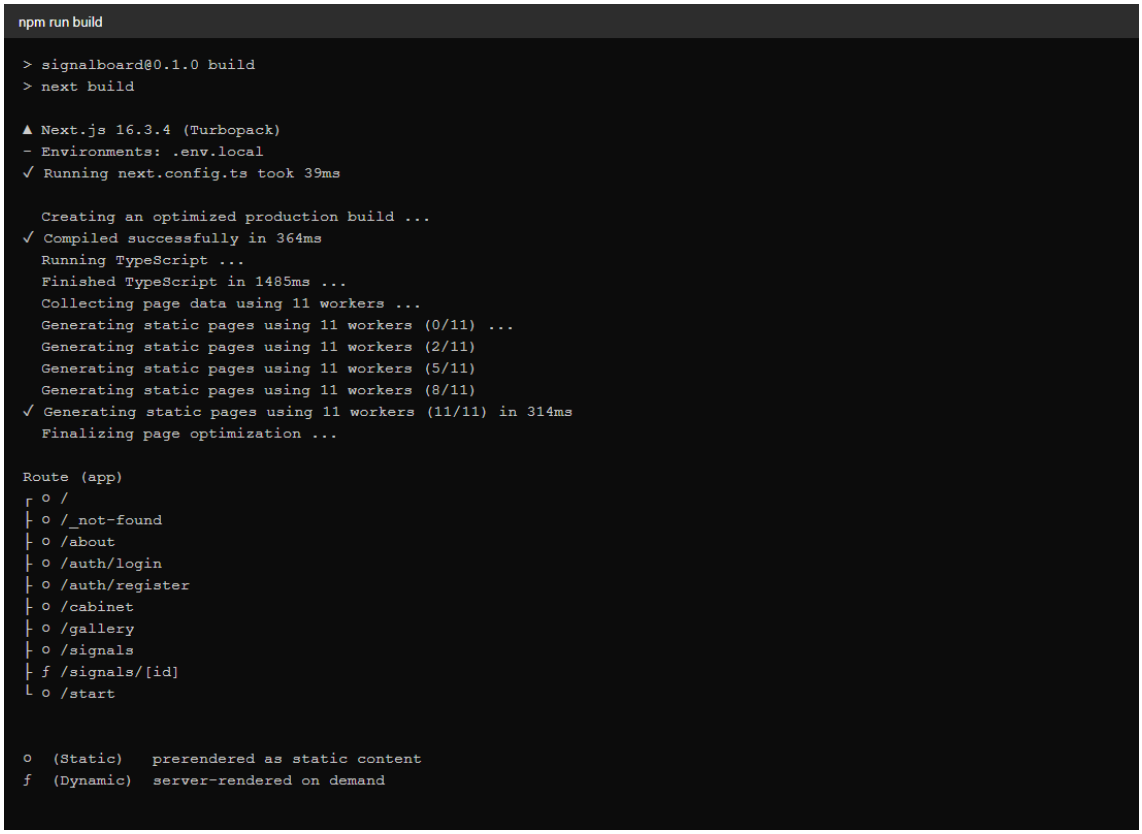


Рисунок 10 – Результат npm run build

4 Вывод

В ходе работы клиентская часть SignalBoard дополнена фотографиями сигналов через next/image, интерактивной галереей, доступным модальным окном на createPortal с управлением фокусом и закрытием по Escape, бургер-меню с активным маршрутом и картой Яндекс Карт с метками, открывающими то же окно. Интерактивность изолирована в Client Components, а страница галереи осталась серверной. Внешние ресурсы освобождаются в cleanup-функциях useEffect. Условное действие «Отслеживать сигнал» реализовано как учебное состояние и явно не считается авторизацией. Проект проходит npm run lint и npm run build.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

Файл app/data/signals.ts

```
// Расширяет локальный сигнал фотографией, альтернативным текстом и
координатами.

// Описываем поля одной предметной сущности.
export type NeighborSignal = {
  id: string;
  title: string;
  summary: string;
  category: string;
  location: string;
  image: string;
  imageAlt: string;
  latitude: number;
  longitude: number;
};

// Храним демонстрационный набор объектов в типизированном массиве.
export const neighborSignals: NeighborSignal[] = [
  {
    id: "lost-keys",
    title: "Ищу ключи от подъезда",
    summary: "Потерял связку ключей у детской площадки вечером, нужна помощь
соседей.",
    category: "ищу",
    location: "Двор дома, детская площадка",
    image: "/images/places/lost-keys.jpg",
    imageAlt: "Детская площадка во дворе, где были потеряны ключи",
    latitude: 59.9318,
    longitude: 30.2985,
  },
  {
    id: "give-chair",
    title: "Отдаю офисный стул",
    summary: "Отдаю исправный стул самовывозом из второго подъезда, забирать
сегодня-завтра.",
    category: "отдаю",
    location: "Второй подъезд, площадка 1 этажа",
    image: "/images/places/give-chair.jpg",
    imageAlt: "Офисный стул у входа во второй подъезд",
    latitude: 59.9326,
    longitude: 30.3012,
  },
  {
    id: "water-outage",
    title: "Отключение воды в доме",
    summary: "УК предупредила: завтра с 10:00 до 16:00 не будет холодной воды
по всему стояку.",
    category: "предупреждаю",
    location: "Весь дом, стояк холодной воды",
    image: "/images/places/water-outage.jpg",
    imageAlt: "Технический стояк холодной воды в подъезде",
    latitude: 59.9334,
    longitude: 30.2968,
  },
];
```

```
export function getNeighborSignal(id: string): NeighborSignal | undefined {
  return neighborSignals.find((signal) => signal.id === id);
}
```

Файл app/gallery/page.tsx

```
// Подключает интерактивную галерею к серверной странице маршрута.

import SignalsGallery from "./_components/SignalsGallery";

// Объявляем основной компонент этого файла.
export default function GalleryPage() {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <main>
      <h1>Галерея сигналов</h1>
      <p>
        Выберите карточку или метку на карте, чтобы открыть подробное окно.
      </p>
      <SignalsGallery />
    </main>
  );
}
```

Файл app/gallery/_components/SignalsGallery.tsx

```
// Управляет карточками, выбранным объектом, отслеживанием, модальным окном и
картой.

"use client";

import Image from "next/image";
import { useCallback, useState } from "react";
import { neighborSignals, type NeighborSignal } from "../../data/signals";
import PhotoMap from "./PhotoMap";
import PhotoModal from "./PhotoModal";
import styles from "./SignalsGallery.module.css";

// Объявляем основной компонент этого файла.
export default function SignalsGallery() {
  // Создаём локальное состояние компонента.
  const [selected, setSelected] = useState<NeighborSignal | null>(null);
  const [trackedIds, setTrackedIds] = useState<Set<string>>(
    () => new Set<string>(),
  );
  const [returnFocusTo, setReturnFocusTo] = useState<HTMLButtonElement |
null>(null);

  function openFromCard(signal: NeighborSignal, button: HTMLButtonElement) {
    setReturnFocusTo(button);
    setSelected(signal);
  }

  const openFromMap = useCallback((signal: NeighborSignal) => {
    setReturnFocusTo(null);
    setSelected(signal);
  }, []);

  function closeModal() {
    setSelected(null);
  }

  function toggleTracked(id: string) {
```

```
    setTrackedIds((current) => {
      const next = new Set(current);
      if (next.has(id)) next.delete(id);
      else next.add(id);
      return next;
    });
  }

  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <>
      <div className={styles.grid}>
        {neighborSignals.map((signal) => (
          <button
            className={styles.card}
            type="button"
            key={signal.id}
            onClick={(event) => openFromCard(signal, event.currentTarget)}
          >
            <span className={styles.imageWrap}>
              <Image
                src={signal.image}
                alt={signal.imageAlt}
                fill
                sizes="(max-width: 700px) 100vw, 33vw"
                className={styles.image}
              />
            </span>
            <span className={styles.title}>{signal.title}</span>
            <span className={styles.summary}>{signal.summary}</span>
          </button>
        ))}
      </div>

      <p className={styles.demoNote}>
        Отслеживание в этой работе хранится только до обновления страницы.
      </p>

      <PhotoMap signals={neighborSignals} onSelect={openFromMap} />

      {selected ? (
        <PhotoModal
          signal={selected}
          isTracked={trackedIds.has(selected.id)}
          onToggleTracked={() => toggleTracked(selected.id)}
          onClose={closeModal}
          returnFocusTo={returnFocusTo}
        />
      ) : null}
    </>
  );
}
```

Файл app/gallery/_components/SignalsGallery.module.css

```
/* Оформляет адаптивную сетку карточек и состояния галереи. */

/* Создаём адаптивную сетку карточек. */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 18px;
  margin-top: 24px;
```

```
}

/* Оформляем интерактивную карточку объекта. */
.card {
  display: grid;
  gap: 10px;
  overflow: hidden;
  padding: 0 0 18px;
  border: 1px solid #c7d4e4;
  border-radius: 14px;
  background: #ffffff;
  color: inherit;
  text-align: left;
}

/* Показываем заметный фокус при клавиатурной навигации. */
.card:focus-visible {
  outline: 4px solid #1d70a2;
  outline-offset: 4px;
}

/* Резервируем область изображения с постоянными пропорциями. */
.imageWrap {
  position: relative;
  display: block;
  aspect-ratio: 3 / 2;
  overflow: hidden;
  background: #d9dde2;
}

.image {
  object-fit: cover;
}

.title,
.summary {
  padding: 0 16px;
}

.title {
  font-size: 20px;
  font-weight: 800;
}

.summary {
  line-height: 1.5;
}

.demoNote {
  margin: 18px 0 32px;
  color: #5b6470;
}
```

Файл app/gallery/_components/PhotoModal.tsx

```
// Создаёт доступное модальное окно с изображением, действиями и управлением
фокусом.

"use client";

import Image from "next/image";
import Link from "next/link";
```

```
import { useEffect, useRef, type KeyboardEvent as ReactKeyboardEvent } from
"react";
import { createPortal } from "react-dom";
import { useDemoAuth } from "../../providers/DemoAuthProvider";
import type { NeighborSignal } from "../../data/signals";
import styles from "./PhotoModal.module.css";

// Типизируем входные свойства компонента.
type PhotoModalProps = {
  signal: NeighborSignal;
  isTracked: boolean;
  onToggleTracked: () => void;
  onClose: () => void;
  returnFocusTo: HTMLElement | null;
};

// Собираем селектор элементов, которые могут получать фокус.
const focusableSelector = [
  "a[href]",
  "button:not([disabled])",
  "input:not([disabled])",
  "select:not([disabled])",
  "textarea:not([disabled])",
  '[tabindex]:not([tabindex="-1"])',
].join(",");

// Объявляем основной компонент этого файла.
export default function PhotoModal({
  signal,
  isTracked,
  onToggleTracked,
  onClose,
  returnFocusTo,
}: PhotoModalProps) {
  const { user } = useDemoAuth();
  const dialogRef = useRef<HTMLDivElement>(null);
  const closeButtonRef = useRef<HTMLButtonElement>(null);

  // Синхронизируем компонент с браузерными событиями или внешним API.
  useEffect(() => {
    const previousOverflow = document.documentElement.style.overflow;
    document.documentElement.style.overflow = "hidden";
    closeButtonRef.current?.focus();

    function handleEscape(event: globalThis.KeyboardEvent) {
      if (event.key === "Escape") onClose();
    }

    window.addEventListener("keydown", handleEscape);

    // В cleanup освобождаем подписки и внешние ресурсы эффекта.
    return () => {
      window.removeEventListener("keydown", handleEscape);
      document.documentElement.style.overflow = previousOverflow;
      returnFocusTo?.focus();
    };
  }, [onClose, returnFocusTo]);

  // Не выпускаем клавиатурный фокус за границы открытого интерфейса.
  function keepFocusInside(event: ReactKeyboardEvent<HTMLDivElement>) {
    if (event.key !== "Tab") return;
```

```
    const focusable = Array.from(
      dialogRef.current?.querySelectorAll<HTMLElement>(focusableSelector) ??
[],
    );

    if (focusable.length === 0) {
      // Отменяем стандартное действие события и выполняем собственную
логику.
      event.preventDefault();
      return;
    }

    const first = focusable[0];
    const last = focusable[focusable.length - 1];

    if (event.shiftKey && document.activeElement === first) {
      event.preventDefault();
      last.focus();
    } else if (!event.shiftKey && document.activeElement === last) {
      event.preventDefault();
      first.focus();
    }
  }

  // Рендерим окно в document.body поверх основного интерфейса.
  return createPortal(
    <div
      className={styles.overlay}
      onMouseDown={(event) => {
        if (event.target === event.currentTarget) onClose();
      }}
    >
      <div
        ref={dialogRef}
        className={styles.dialog}
        role="dialog"
        aria-modal="true"
        aria-labelledby="signal-modal-title"
        onKeyDown={keepFocusInside}
      >
        <button
          ref={closeButtonRef}
          className={styles.close}
          type="button"
          aria-label="Закрыть окно"
          onClick={onClose}
        >
          ×
        </button>

        <h2 id="signal-modal-title">{signal.title}</h2>

        <Image
          src={signal.image}
          alt={signal.imageAlt}
          width={900}
          height={600}
          className={styles.image}
          sizes="(max-width: 700px) calc(100vw - 48px), 720px"
        />
```

```
        <p>{signal.summary}</p>
        <p>
          <strong>Тип:</strong> {signal.category}
        </p>
        <p>
          <strong>Место:</strong> {signal.location}
        </p>

        {user ? (
          <button type="button" onClick={onToggleTracked}>
            {isTracked ? "Не отслеживать сигнал" : "Отслеживать сигнал"}
          </button>
        ) : (
          <Link href="/auth/login">Войдите, чтобы отслеживать сигнал</Link>
        )}
      </div>
    </div>,
    document.body,
  );
}
```

Файл app/gallery/_components/PhotoModal.module.css

```
/* Размещает модальное окно поверх страницы и ограничивает его видимой
областью. */

/* Перекрываем viewport затемнённым фоном. */
.overlay {
  position: fixed;
  inset: 0;
  z-index: 3000;
  display: grid;
  place-items: center;
  padding: 24px;
  background: rgb(15 23 42 / 72%);
}

/* Ограничиваем размеры окна и разрешаем прокрутку длинного содержимого. */
.dialog {
  position: relative;
  width: min(760px, 100%);
  max-height: calc(100dvh - 48px);
  overflow-y: auto;
  padding: 28px;
  border-radius: 16px;
  background: #ffffff;
  box-shadow: 0 24px 70px rgb(0 0 0 / 35%);
}

.dialog h2 {
  margin: 0 48px 18px 0;
}

.close {
  position: absolute;
  top: 14px;
  right: 14px;
  width: 40px;
  height: 40px;
  border: 0;
  border-radius: 50%;
```

```
  background: #e8edf2;
  font-size: 28px;
  line-height: 1;
}

/* Показываем заметный фокус при клавиатурной навигации. */
.close:focus-visible,
.dialog a:focus-visible,
.dialog button:focus-visible {
  outline: 4px solid #1d70a2;
  outline-offset: 3px;
}

.image {
  width: 100%;
  height: auto;
  border-radius: 12px;
  object-fit: cover;
}
```

Файл app/gallery/_components/PhotoMap.tsx

```
// Загружает API Яндекс Карт, создаёт карту и связывает метки с объектами.

"use client";

import { useEffect, useRef, useState } from "react";
import type { NeighborSignal } from "../../data/signals";
import styles from "./PhotoMap.module.css";

// Координаты представлены парой широты и долготы.
type Coordinates = [number, number];

type YandexPlacemark = {
  events: {
    add: (eventName: string, handler: () => void) => void;
  };
};

type YandexMap = {
  geoObjects: {
    add: (placemark: YandexPlacemark) => void;
  };
  destroy: () => void;
};

type YandexMapsApi = {
  ready: (callback: () => void) => void;
  Map: new (
    container: HTMLElement,
    state: { center: Coordinates; zoom: number; controls: string[] },
  ) => YandexMap;
  Placemark: new (
    coordinates: Coordinates,
    properties: { hintContent: string; balloonContent: string },
    options: { preset: string; iconColor: string },
  ) => YandexPlacemark;
};

// Дополняем тип Window объектом, который добавляет внешний API.
declare global {
  interface Window {
```

```
    ymaps?: YandexMapsApi;
  }
}

const scriptId = "signalboard-yandex-maps";
let mapsPromise: Promise<YandexMapsApi> | null = null;

// Загружаем скрипт карты один раз и ожидаем готовности API.
function loadMaps(apiKey: string): Promise<YandexMapsApi> {
  if (window.ymaps) {
    return new Promise((resolve) => {
      window.ymaps?.ready(() => resolve(window.ymaps as YandexMapsApi));
    });
  }

  if (mapsPromise) return mapsPromise;

  mapsPromise = new Promise((resolve, reject) => {
    function resolveWhenReady() {
      if (!window.ymaps) {
        mapsPromise = null;
        reject(new Error("Yandex Maps API is unavailable"));
        return;
      }

      window.ymaps.ready(() => resolve(window.ymaps as YandexMapsApi));
    }

    const existing = document.getElementById(scriptId) as HTMLScriptElement |
null;
    if (existing) {
      existing.addEventListener("load", resolveWhenReady, { once: true });
      existing.addEventListener("error", () => reject(new Error("Map script
error")), {
        once: true,
      });
      return;
    }

    const script = document.createElement("script");
    script.id = scriptId;
    script.src = `https://api-
maps.yandex.ru/2.1/?apikey=${encodeURIComponent(apiKey)}&lang=ru_RU`;
    script.async = true;
    script.addEventListener("load", resolveWhenReady, { once: true });
    script.addEventListener(
      "error",
      () => {
        mapsPromise = null;
        reject(new Error("Map script error"));
      },
      { once: true },
    );
    // Подключаем внешний скрипт к документу.
    document.head.appendChild(script);
  });

  return mapsPromise;
}

// Объявляем основной компонент этого файла.
```

```
export default function PhotoMap({
  signals,
  onSelect,
}: {
  signals: NeighborSignal[];
  onSelect: (signal: NeighborSignal) => void;
}) {
  const containerRef = useRef<HTMLDivElement>(null);
  // Создаём локальное состояние компонента.
  const apiKey = process.env.NEXT_PUBLIC_YANDEX_MAPS_API_KEY ?? "";
  const [status, setStatus] = useState<"loading" | "ready" | "error" | "no-
key">(
    apiKey ? "loading" : "no-key",
  );

  // Синхронизируем компонент с браузерными событиями или внешним API.
  useEffect(() => {
    if (!apiKey) {
      return;
    }

    if (!containerRef.current || signals.length === 0) return;

    let disposed = false;
    let map: YandexMap | null = null;

    async function buildMap() {
      try {
        const ymaps = await loadMaps(apiKey);
        if (disposed || !containerRef.current) return;

        const first = signals[0];
        // Создаём экземпляр карты в подготовленном DOM-контейнере.
        map = new ymaps.Map(containerRef.current, {
          center: [first.latitude, first.longitude],
          zoom: 15,
          controls: ["zoomControl"],
        });

        // Добавляем на карту метку для каждого сигнала.
        signals.forEach((signal) => {
          // Создаём метку с координатами и данными текущего объекта.
          const placemark = new ymaps.Placemark(
            [signal.latitude, signal.longitude],
            {
              hintContent: signal.title,
              balloonContent: signal.location,
            },
            {
              preset: "islands#circleDotIcon",
              iconColor: "#134f9c",
            },
          );

          placemark.events.add("click", () => onSelect(signal));
          map?.geoObjects.add(placemark);
        });

        setStatus("ready");
      } catch {
        if (!disposed) setStatus("error");
```

```
      }
    }

    void buildMap();

    // В cleanup освобождаем подписки и внешние ресурсы эффекта.
    return () => {
      disposed = true;
      // Уничтожаем экземпляр карты и его обработчики.
      map?.destroy();
    };
  }, [apiKey, onSelect, signals]);

  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <section className={styles.section} aria-labelledby="map-title">
      <h2 id="map-title">Карта сигналов двора</h2>

      {status === "no-key" ? (
        <p className={styles.status}>
          Добавьте NEXT_PUBLIC_YANDEX_MAPS_API_KEY в файл .env.local.
        </p>
      ) : null}

      {status === "error" ? (
        <p className={styles.status}>Не удалось загрузить карту.</p>
      ) : null}

      {status === "loading" ? (
        <p className={styles.status}>Карта загружается…</p>
      ) : null}

      <div
        ref={containerRef}
        className={styles.canvas}
        aria-label="Интерактивная карта локальных сигналов"
      />
    </section>
  );
}
```

Файл app/gallery/_components/PhotoMap.module.css

```
/* Задаёт размеры контейнера карты и оформление сообщений состояния. */

.section {
  margin-top: 48px;
}

/* Задаём явные размеры контейнера интерактивной карты. */
.canvas {
  width: 100%;
  height: 480px;
  overflow: hidden;
  border: 1px solid #b9c0c8;
  border-radius: 16px;
  background: #dbe3ea;
}

/* Выделяем сообщения о загрузке или ошибке карты. */
.status {
  padding: 12px 14px;
```

```
  border-left: 4px solid #134f9c;
  background: #e9f2f8;
}
```

Файл app/ui/BurgerMenu/BurgerMenu.tsx

```
// Реализует доступное бургер-меню с маршрутами и состоянием пользователя.

"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import {
  useEffect,
  useRef,
  useState,
  type KeyboardEvent as ReactKeyboardEvent,
} from "react";
import { useDemoAuth } from "../../providers/DemoAuthProvider";
import styles from "./styles.module.css";

// Описываем пункты меню единым массивом.
const links = [
  { href: "/start", label: "Главная" },
  { href: "/signals", label: "Лента" },
  { href: "/gallery", label: "Галерея и карта" },
  { href: "/about", label: "О проекте" },
  { href: "/cabinet", label: "Кабинет" },
];

// Собираем селектор доступных элементов внутри меню.
const menuFocusableSelector = [
  "a[href]",
  "button:not([disabled])",
  '[tabindex]:not([tabindex="-1"])',
].join(",");

function isActive(pathname: string, href: string): boolean {
  return pathname === href || pathname.startsWith(`${href}/`);
}

// Объявляем основной компонент этого файла.
export default function BurgerMenu() {
  const pathname = usePathname();
  const { user, logout } = useDemoAuth();
  // Создаём локальное состояние компонента.
  const [openedOn, setOpenedOn] = useState<string | null>(null);
  // Панель открыта только на маршруте, где её открыли: переход по ссылке
закрывает меню.
  const open = openedOn === pathname;
  const openButtonRef = useRef<HTMLButtonElement>(null);
  const closeButtonRef = useRef<HTMLButtonElement>(null);
  const panelRef = useRef<HTMLElement>(null);
  const ignoreCloseRef = useRef(false);

  // Синхронизируем компонент с браузерными событиями или внешним API.
  useEffect(() => {
    if (!open) return;

    const previousOverflow = document.documentElement.style.overflow;
    const openButton = openButtonRef.current;
    document.documentElement.style.overflow = "hidden";
```

```
    closeButtonRef.current?.focus();

    function handleEscape(event: KeyboardEvent) {
      if (event.key === "Escape") setOpenedOn(null);
    }

    window.addEventListener("keydown", handleEscape);

    // В cleanup освобождаем подписки и внешние ресурсы эффекта.
    return () => {
      window.removeEventListener("keydown", handleEscape);
      document.documentElement.style.overflow = previousOverflow;
      openButton?.focus();
    };
  }, [open]);

  function openMenu() {
    ignoreCloseRef.current = true;
    setOpenedOn(pathname);
    window.setTimeout(() => {
      ignoreCloseRef.current = false;
    }, 400);
  }

  function closeMenu() {
    setOpenedOn(null);
  }

  function closeMenuIfSettled() {
    if (ignoreCloseRef.current) return;
    setOpenedOn(null);
  }

  // Не выпускаем клавиатурный фокус за границы открытого интерфейса.
  function keepFocusInside(event: ReactKeyboardEvent<HTMLElement>) {
    if (event.key !== "Tab") return;

    const focusable = Array.from(
      panelRef.current?.querySelectorAll<HTMLElement>(menuFocusableSelector)
?? [],
    );
    if (focusable.length === 0) {
      // Отменяем стандартное действие события и выполняем собственную
логику.
      event.preventDefault();
      return;
    }

    const first = focusable[0];
    const last = focusable[focusable.length - 1];
    if (event.shiftKey && document.activeElement === first) {
      event.preventDefault();
      last.focus();
    } else if (!event.shiftKey && document.activeElement === last) {
      event.preventDefault();
      first.focus();
    }
  }

  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
```

```
    <>
      <button
        ref={openButtonRef}
        className={styles.openButton}
        type="button"
        aria-label="Открыть меню"
        aria-expanded={open}
        aria-controls="main-menu-panel"
        aria-hidden={open}
        tabIndex={open ? -1 : undefined}
        onClick={(event) => {
          event.stopPropagation();
          openMenu();
        }}
      >
        <span aria-hidden="true">☰</span>
      </button>

      {open ? (
        <div
          className={styles.overlay}
          onMouseDown={(event) => {
            if (event.target === event.currentTarget) closeMenuIfSettled();
          }}
        >
          <section
            ref={panelRef}
            id="main-menu-panel"
            className={styles.panel}
            role="dialog"
            aria-modal="true"
            aria-label="Основное меню"
            onKeyDown={keepFocusInside}
          >
            <div className={styles.header}>
              <Link href="/start" onClick={closeMenu}>
                SignalBoard
              </Link>
              <button
                ref={closeButtonRef}
                type="button"
                aria-label="Закрыть меню"
                onClick={closeMenuIfSettled}
              >
                ×
              </button>
            </div>

            <nav aria-label="Разделы приложения">
              <ul className={styles.list}>
                {links.map((link) => (
                  <li key={link.href}>
                    <Link
                      href={link.href}
                      aria-current={isActive(pathname, link.href) ? "page" :
undefined}
                      onClick={closeMenu}
                    >
                      {link.label}
                    </Link>
                  </li>
                ))}
```

```
            </ul>
          </nav>

          <div className={styles.authRow}>
            {user ? (
              <>
                <span>Пользователь: {user.name}</span>
                <button
                  type="button"
                  onClick={() => {
                    logout();
                    closeMenu();
                  }}
                >
                  Выйти
                </button>
              </>
            ) : (
              <Link href="/auth/login" onClick={closeMenu}>
                Войти / зарегистрироваться
              </Link>
            )}
          </div>
        </section>
      </div>
    ) : null}
  </>
 );
}
```

Файл app/ui/BurgerMenu/styles.module.css

```
/* Оформляет кнопку меню, затемнение и выезжающую панель навигации. */

/* Закрепляем кнопку открытия меню поверх страницы. */
.openButton {
  position: fixed;
  top: 16px;
  right: 16px;
  z-index: 2000;
  display: grid;
  width: 48px;
  height: 48px;
  place-items: center;
  border: 1px solid #b9c0c8;
  border-radius: 50%;
  background: #ffffff;
  font-size: 24px;
  cursor: pointer;
  pointer-events: auto;
  touch-action: manipulation;
  box-shadow: 0 4px 16px rgb(15 23 42 / 16%);
}

/* Скрываем кнопку открытия, пока панель уже на экране: иначе × стоит в тех
же пикселях. */
.openButton[aria-expanded="true"] {
  visibility: hidden;
  pointer-events: none;
}

/* Показываем заметный фокус при клавиатурной навигации. */
.openButton:focus-visible,
```

```
.panel a:focus-visible,
.panel button:focus-visible {
  outline: 4px solid #1d70a2;
  outline-offset: 3px;
}

/* Перекрываем viewport затемнённым фоном. */
.overlay {
  position: fixed;
  inset: 0;
  z-index: 2200;
  display: flex;
  justify-content: flex-end;
  background: rgb(15 23 42 / 55%);
}

/* Размещаем панель меню у правого края viewport. */
.panel {
  width: min(430px, 100%);
  min-height: 100%;
  margin-left: auto;
  padding: 24px;
  background: #ffffff;
  box-shadow: -20px 0 50px rgb(0 0 0 / 25%);
}

.header,
.authRow {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
}

.header a {
  color: #1f2933;
  font-size: 24px;
  font-weight: 800;
  text-decoration: none;
}

.header button {
  width: 42px;
  height: 42px;
  border: 0;
  border-radius: 50%;
  font-size: 28px;
}

.list {
  display: grid;
  gap: 10px;
  margin: 48px 0;
  padding: 0;
  list-style: none;
}

.list a {
  display: block;
  padding: 12px;
  border-radius: 10px;
```

```
  color: #1f2933;
  font-size: 20px;
  text-decoration: none;
}

.list a[aria-current="page"] {
  background: #dbeafe;
  color: #123b5d;
  font-weight: 800;
}

.authRow {
  padding-top: 20px;
  border-top: 1px solid #c7d4e4;
}
```

Файл app/layout.tsx

```
// Подключает бургер-меню и учебный провайдер к корневому layout.

import type { Metadata } from "next";
import type { ReactNode } from "react";
import DemoAuthProvider from "./providers/DemoAuthProvider";
import BurgerMenu from "./ui/BurgerMenu/BurgerMenu";
import "./globals.css";

// Задаём заголовок и описание страницы для Metadata API.
export const metadata: Metadata = {
  title: "SignalBoard",
  description: "Доска локальных сигналов соседям",
};

// Объявляем основной компонент этого файла.
export default function RootLayout({
  children,
}: Readonly<{ children: ReactNode }>) {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <html lang="ru">
      <body>
        <DemoAuthProvider>
          <BurgerMenu />
          {children}
        </DemoAuthProvider>
      </body>
    </html>
  );
}
```